

TP INTEGRADOR · ACTIVIDAD 4
Grupo 3 · S42

Preparándonos para Producción.

Ingeniería y Calidad de Software · 2026
Universidad Tecnológica Nacional – FRLP

26980	Marcelo	Tassi
31890	Llontop	Alejandro
27978	Delozano	Matias
25619	Di Bella	Abel
33217	Lafalce	Mateo

— INTRO —

Las cuatro fases

Analizar → **Diseñar** → **Implementar** → **Verificar**

-
- | | |
|-----------------|---|
| 1 · Analizar | problemas de la infra Docker actual + investigar OpenTelemetry. |
| 2 · Diseñar | multi-stage builds, compose de producción y dashboard RED. |
| 3 · Implementar | Dockerfiles seguros, API instrumentada, Prometheus + Grafana. |
| 4 · Verificar | seguridad (no-root, read-only) y métricas en vivo. |

— FASE 1 · ANÁLISIS —

5 problemas × 5 integrantes**Infra Docker de desarrollo – hallazgos por integrante****Marcelo Tassi**

- x Sin multi-stage – imagen ~1 GB
- x Corre como root (ningún USER)
- x Credenciales 'password123' en repo
- x Sin límites ni healthchecks
- x 'COPY . .' invalida la caché npm

Alejandro Llontop

- x Sin multi-stage (devDeps + tools)
- x Contenedores como root (sin USER)
- x Credenciales en docker-compose
- x Sin límites ni healthchecks
- x 'migrate dev' resetea datos en prod

Matías De Lozano

- x Falta .dockerignore (.git, .env)
- x Env vars hardcodeadas en compose
- x Volumen de código desprotegido
- x Sin límites ni healthchecks
- x Entrypoint sin '&&' (sigue si falla)

Abel Di Bella

- x Credenciales en docker-compose
- x Ejecución como root en ambos
- x Bind-mounts usados en producción
- x Sin límites de CPU ni memoria
- x Sin multi-stage – más superficie

Mateo Lafalce

- x Imagen monolítica sin multi-stage
- x Credenciales versionadas en repo
- x Procesos como root (uid 0)
- x Bind-mount '.: /app' expone el repo
- x Sin límites ni healthchecks

— FASE 1 · ANÁLISIS —

OpenTelemetry, en una slide

Qué es estándar abierto que genera, recolecta y exporta telemetría
Desacopla instrumentación del backend – mismo código → Prometheus, Jaeger...

3 pilares de la observabilidad

→ Trazas · Métricas · Logs (OTel cubre los tres)

Método RED (Tom Wilkie)

- Rate – requests por segundo (volumen de tráfico)
- Errors – respuestas 4xx/5xx (fallos al usuario)
- Duration – latencia de cada request (performance percibida)

OTLP protocolo que evita acoplar el código a un backend
Grafana capa de visualización: consulta a Prometheus vía PromQL

— DISEÑO + IMPLEMENTACIÓN —

Multi-stage build – API

packages/api/Dockerfile.prod – 3 etapas

```
1 · build      node:22-alpine – npm ci, prisma generate, compila TS → /dist
2 · deps-prod  node:22-alpine – npm ci --omit=dev + poda de paquetes inútiles
3 · runtime    node:22-alpine – solo JS compilado + deps prod, usuario 'node'
```

-
- ✓ npm y npx eliminados del runtime – no quedan herramientas de build.
 - ✓ Engines de Prisma copiadas del stage build (sin postinstall).
 - ✓ Migraciones (migrate deploy) se corren al arrancar, llamando al binario directo.
 - ✓ Healthcheck wget contra 127.0.0.1:3000 cada 30s.

— DISEÑO + IMPLEMENTACIÓN —

Multi-stage build – Web

packages/web/Dockerfile.prod – 3 etapas

- 1 · deps node:22-alpine – npm ci (necesita vite, typescript para compilar)
- 2 · build node:22-alpine – vite build → estáticos en /dist
- 3 · runtime nginx-unprivileged:stable-alpine – sirve los estáticos (uid 101)

-
- ✓ Sin Node.js en producción: nginx sirve la SPA, liviano y rápido.
 - ✓ nginx.conf propio: SPA fallback, gzip, cache de assets y security headers.

— RESULTADOS —

Antes y después: tamaño de imágenes

API	1.67 GB	→	473 MB	(71.7% ↓)
Web	1.02 GB	→	82.5 MB	(91.9% ↓)

Cómo se logró

- Multi-stage: la imagen final no incluye el toolchain de build.
 - Web sin Node: solo estáticos + nginx-unprivileged (base ~50 MB).
- Meta de la consigna: reducción $\geq 70\%$.

— RESULTADOS —

Seguridad aplicada (y por qué)

- ✓ Usuario no-root – API como 'node', Web como nginx uid 101.
- ✓ read_only: true – filesystem inmutable en runtime.
- ✓ no-new-privileges: true – bloquea escalada vía binarios setuid.
- ✓ Sin npm/npx/tsc en la imagen final – menor superficie de ataque.
- ✓ Secrets vía .env con \${VAR} – nada hardcodeado ni versionado.

— IMPLEMENTACIÓN —

`docker-compose.prod.yml`

Cinco servicios: db · api · web · prometheus · grafana

Resource limits	db 0.5cpu/512M · api 1.0cpu/512M · web 0.5cpu/256M
Healthchecks	los tres servicios (pg_isready / wget) cada 30s
Logging	json-file con rotación (max-size 10m, max-file 3)
Red	bridge interna 'alentapp-network'
Secrets	DATABASE_URL, POSTGRES_* desde .env
Orden	api espera a db healthy (depends_on: condition)

→ Reúne todas las medidas de seguridad de la slide anterior por servicio.

— OBSERVABILIDAD —

OpenTelemetry en la API

`packages/api/src/infrastructure/telemetry.ts`

```
const prometheusExporter = new PrometheusExporter({
  port: 9464, endpoint: '/metrics',
});
const sdk = new NodeSDK({
  metricReader: prometheusExporter,
  instrumentations: [ getNodeAutoInstrumentations({ http, fastify }) ],
});
sdk.start();
```

- ✓ Se inicializa ANTES de cualquier otro import (en app.ts).
- ✓ Auto-instrumenta HTTP y Fastify; ignora el propio /metrics.
- ✓ Expone métricas en formato Prometheus en :9464/metrics.

— OBSERVABILIDAD —

Métricas RED capturadas

Rate	<code>http.requests.total</code>	(Counter)	labels: method, route, status
Errors	<code>http.requests.errors</code>	(Counter)	labels: method, route, status
Duration	<code>http.request.duration</code>	(Histogram)	labels: method, route

Además, dos gauges

- `process.memory.usage` – heap del proceso (ObservableGauge).
- `http.requests.active` – requests concurrentes (UpDownCounter).

→ Instrumentación manual en cada handler: `add()` en éxito/error y `record()` de duración.

— OBSERVABILIDAD —

Dashboard RED – Alentapp API

App + OTel SDK → :9464/metrics → Prometheus → Grafana

6 paneles

- 1 · Requests por segundo (time series)
- 2 · Tasa de error % (time series)
- 3 · Latencia p95 / p99 (time series)
- 4 · Por status code (stacked area)
- 5 · Memoria del proceso (time series)
- 6 · Endpoints más lentos (top 5) (bar chart)

— FASE 4 · VERIFICACIÓN —

Verificación

- ✓ `docker compose -f docker-compose.prod.yml up -d`
- ✓ `docker compose -f docker-compose.prod.yml ps`
- ✓ `curl http://localhost/`

Observabilidad

- ✓ `curl http://localhost:9464/metrics` expone `http_requests_total` y `http_request_duration`.
- ✓ Prometheus scrapea :9464 correctamente.
- ✓ Los 6 paneles del dashboard responden al tráfico generado.
- ✓ Las métricas de error reflejan los 4xx/5xx.

— CIERRE —

Lecciones aprendidas

- Sin multi-stage: imágenes monolíticas de ~1 GB.
- Contenedores corriendo como root (sin USER).
- Credenciales hardcodeadas y versionadas en el repo.
- Sin límites de recursos ni healthchecks.
- Bind-mounts / volúmenes de código usados en producción.

Qué cambiaríamos

- Usar versiones fijas e inmutables en los Dockerfiles:
- en vez de node:22-alpine usar node:22.14.0-alpine3.21.

¡GRACIAS!

TP Integrador · Actividad 4 – Preparando para Producción
Grupo 3 · S42

26980	Marcelo	Tassi
31890	Llontop	Alejandro
27978	Delozano	Matias
25619	Di Bella	Abel
33217	Lafalce	Mateo